

COMP3311

Week 3 Tutorial



Agenda

✓ Assignment 1 style tips

✓ Week 2 key concepts

✓ Tutorial questions

Assignment 1

style tips

```
# bad >>>:(  
SELECT  
    name  
    age  
FROM      table  
WHERE  
    age < 50
```

```
# good :)  
SELECT  
    name  
    age  
FROM  
    table  
WHERE  
    age < 50
```

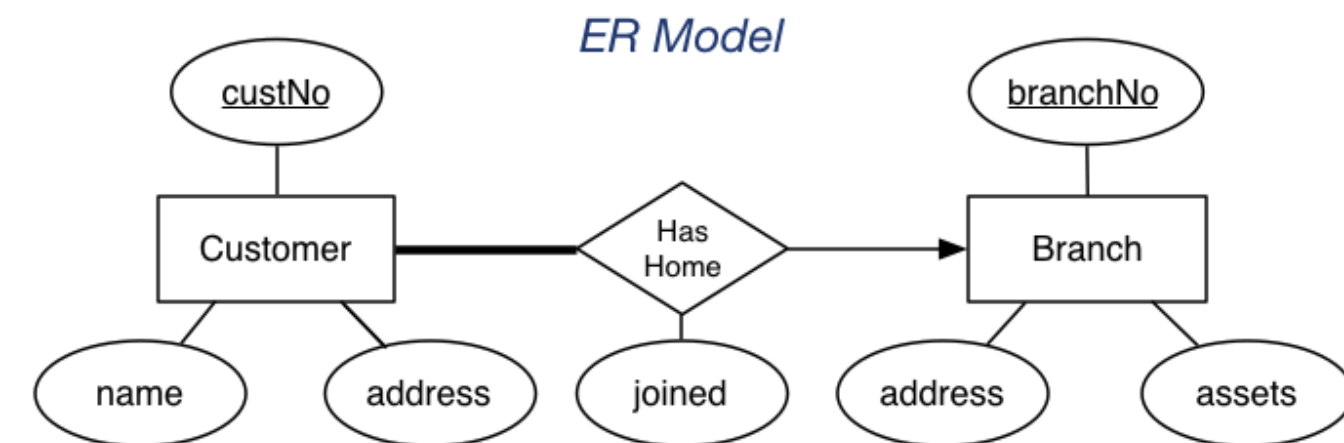
- apply good style conventions you would've learnt in previous courses such as:
 - consistent indentation
 - consistent capitalisation (either capitalised keywords or non-capitalised)
 - avoid long lines (100-120 characters max)
 - avoid repetition (use helper views or helper functions)
- this course has no official style guide but if you are someone that really wants to follow a style guide you can follow this one (note: not officially endorsed by this course)

Week 2 Key Concepts

ER → Relational Model: Mapping Relationships

- **Many-to-one**

- Since Customer has **maximum one** home branch, the Customer table can represent the relationship via a foreign key referencing the Branch's primary key.
- Customer then also inherits the relationship attribute joined as well



Relational Version

Customer	custNo	name	address	branchNo	joined
Branch	branchNo	address	assets		

Week 2 Key Concepts

ER → Relational Model: Mapping Relationships

e.g.

With foreign key in Branch:

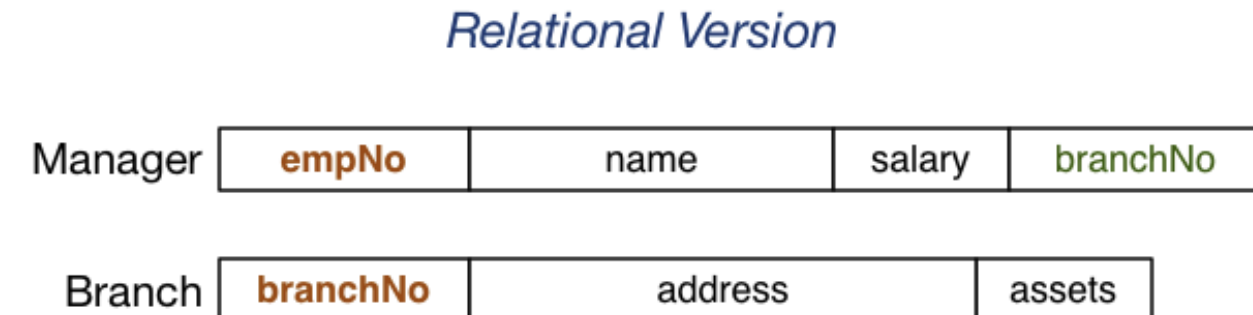
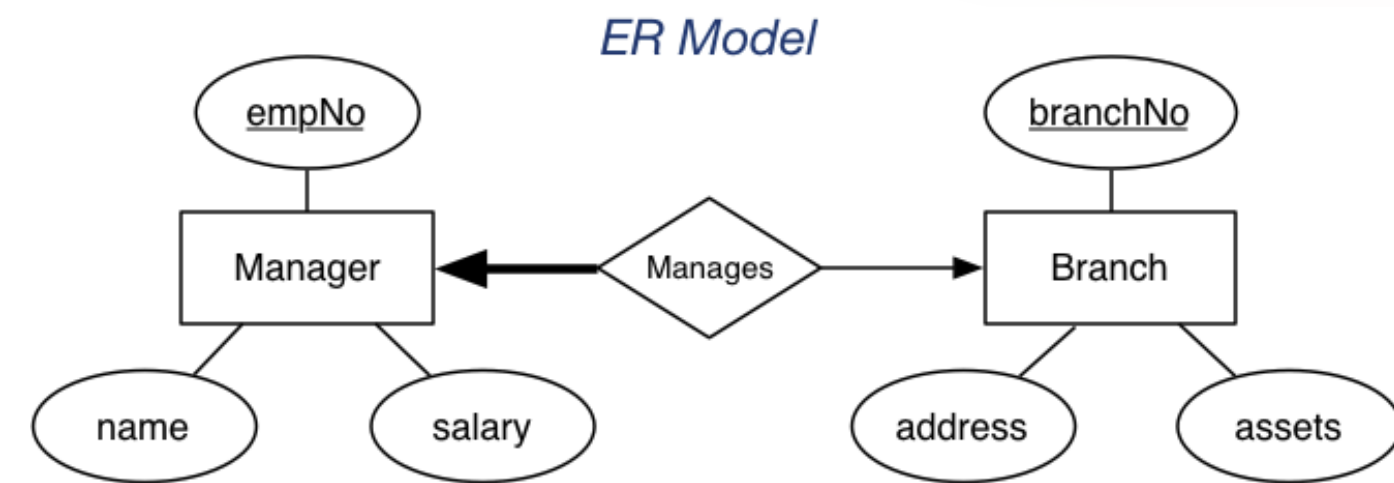
- Branch(1, 123 Smith Street, 2000, empNo1)
- Branch(2, 46 High Street, 3000, **NULL**)
- Branch(3, 58 Anzac Parade, 4000, **NULL**)

With foreign key in Manager:

- Manager(empNo1, Nicole, 20000, branchNo1)
- Manager(empNo2, Nicholas, 30000, branchNo2)
- Manager(empNo3, Nikkita, 10000, branchNo3)

- **One-to-one**

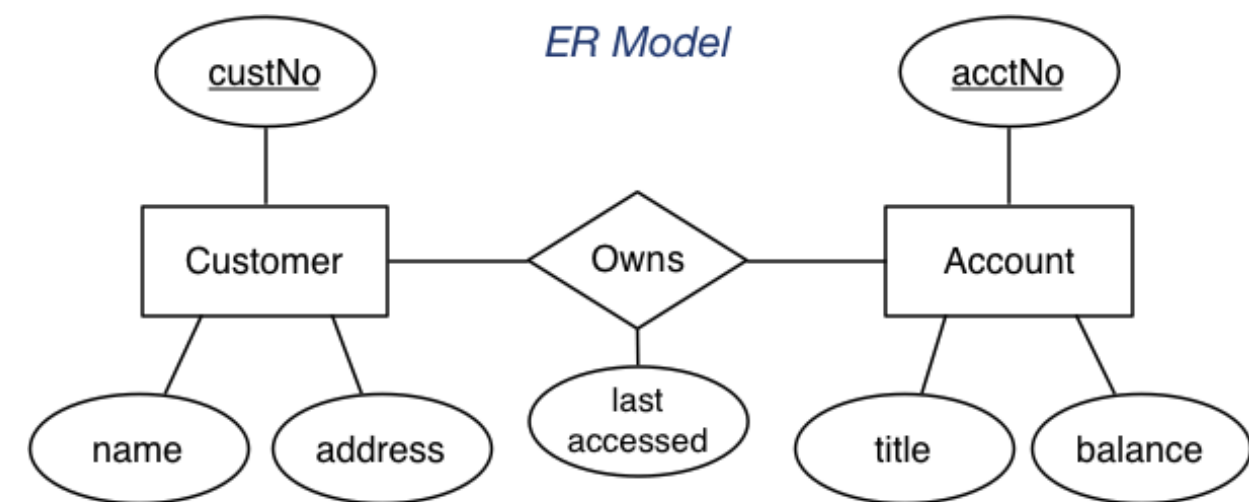
- You could place a foreign key in either entity relation, but Manager is **totally participating** (so a manager must have a branch to manage). This implies $\#managers \leq \#branches$, so **space is saved** by representing the relationship inside Manager.



Week 2 Key Concepts

ER → Relational Model: Mapping Relationships

- **Many-to-many**
 - A **third table** representing the many-to-many relationship is required.
 - This table will have the relationship attribute(s), as well as two foreign keys, each referencing the Entity it's associated with.
 - The **primary key** of this new table is the **combination of both foreign keys** to avoid duplicate relationship instances.



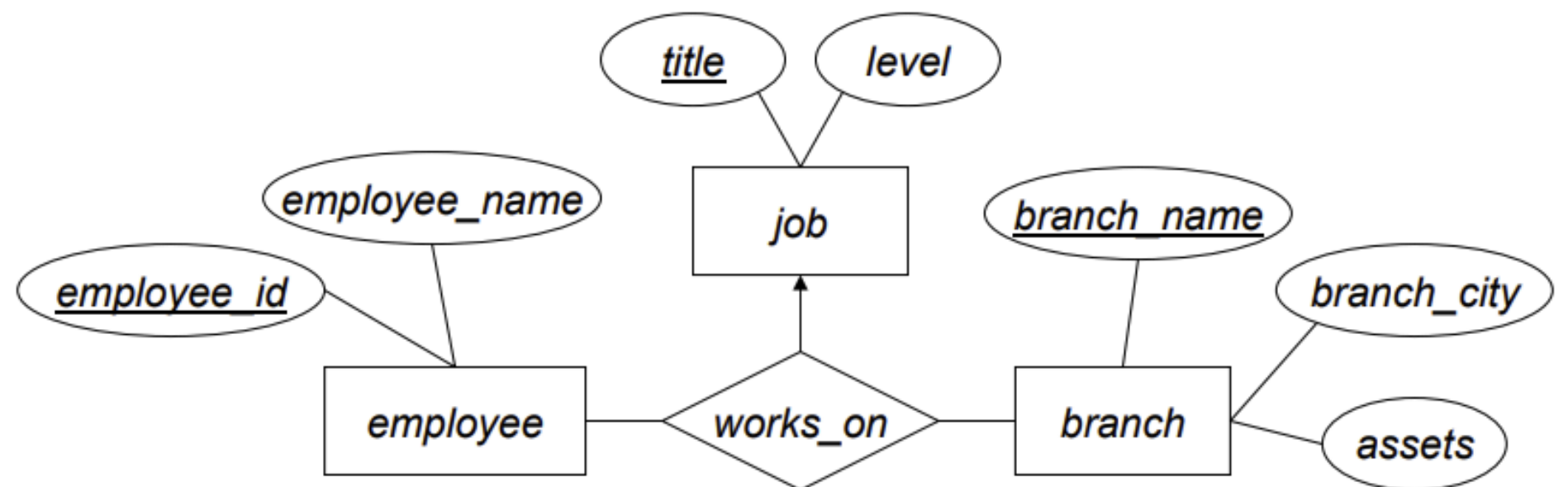
Week 2 Key Concepts

ER → Relational Model: Mapping Relationships

job(title, level)
employee(employee_id, employee_name)
branch(branch_name, branch_city, assets)
works_on(employee_id, branch_name, title)

Since there is one arrow, the Works_On relationship table has the primary key exclude the Job table's foreign key from the primary key of Works_On.

- **N-way relationships**
 - A separate table is required to model the relationship
 - **If no-arrows (many-to-many-to-many):**
 - primary key for the relationship is the union of all participating entity-sets' primary keys.
 - **If one-arrow (one-to-many):**
 - primary key for the relationship is the union of primary keys of entity-sets without an arrow
 - **More than one arrow isn't great**, might be better to redesign the ER diagram

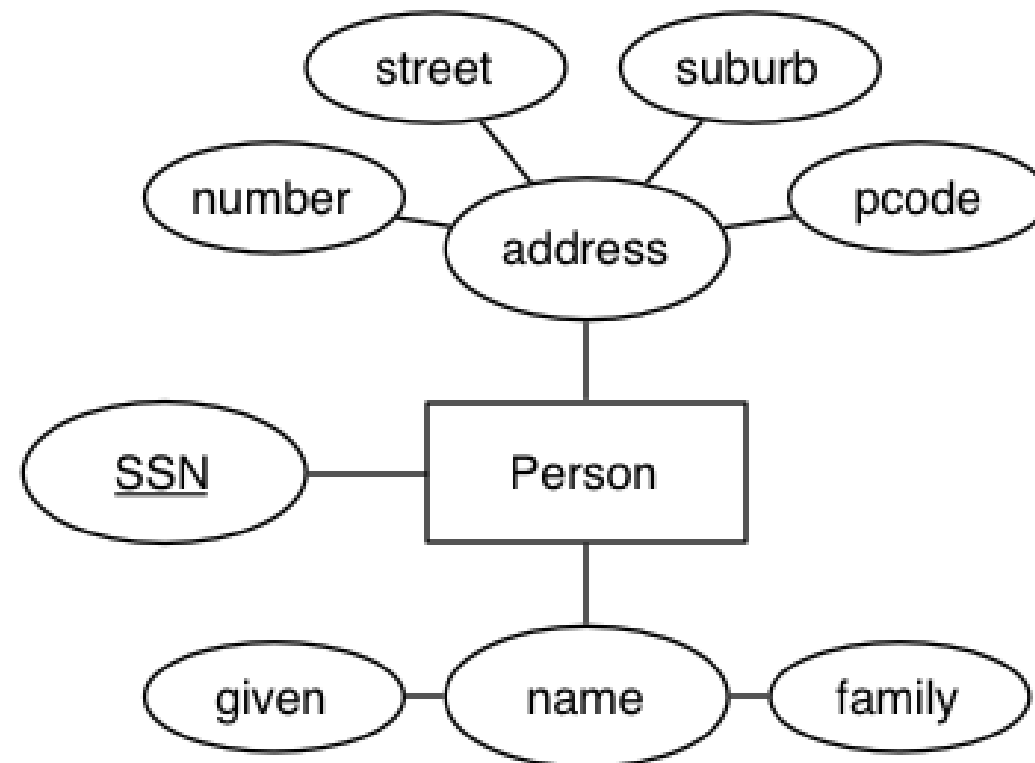


Week 2 Key Concepts

ER → Relational Model: Mapping Relationships

- **Composite attributes**
 - map them either by concatenation, or by flattening.

ER Model



Relational Version #1

Person		
SSN	name	address

Relational Version #2

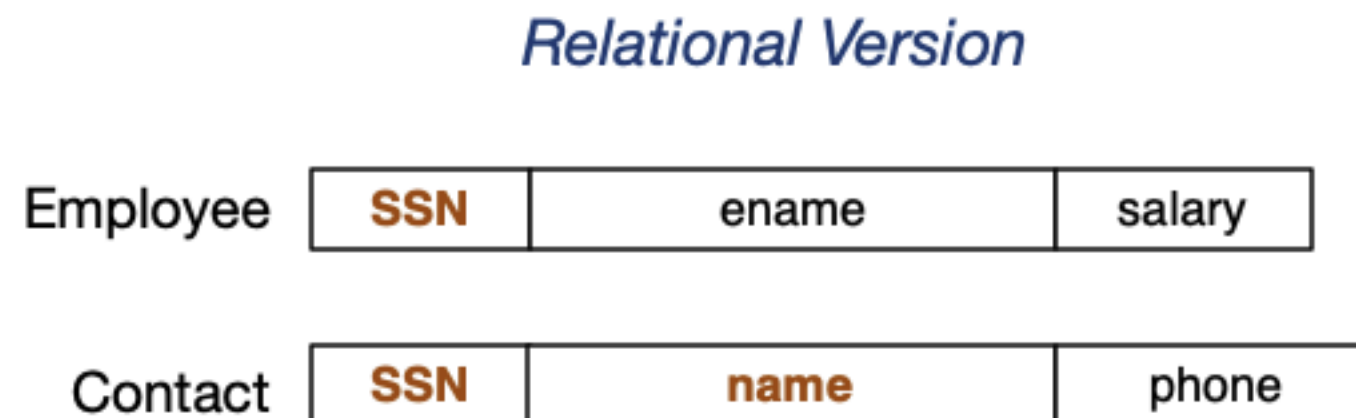
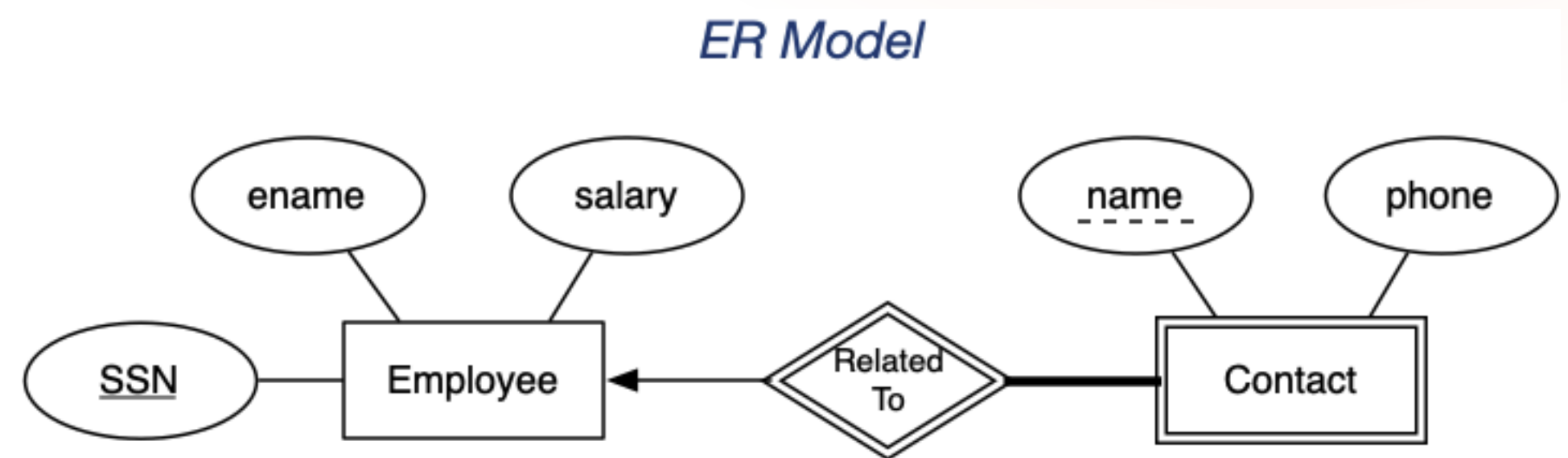
Person				
SSN	given	family		
.....	number	street	suburb	pcode

Week 2 Key Concepts

ER → Relational Model: Mapping Relationships

- **Weak entities**

- For mapping a weak entity, make a **table/relation for the weak entity**, add a **foreign key** referencing the strong entity's primary key, and make the **primary key** of the new table (**foreign key + discriminator**)

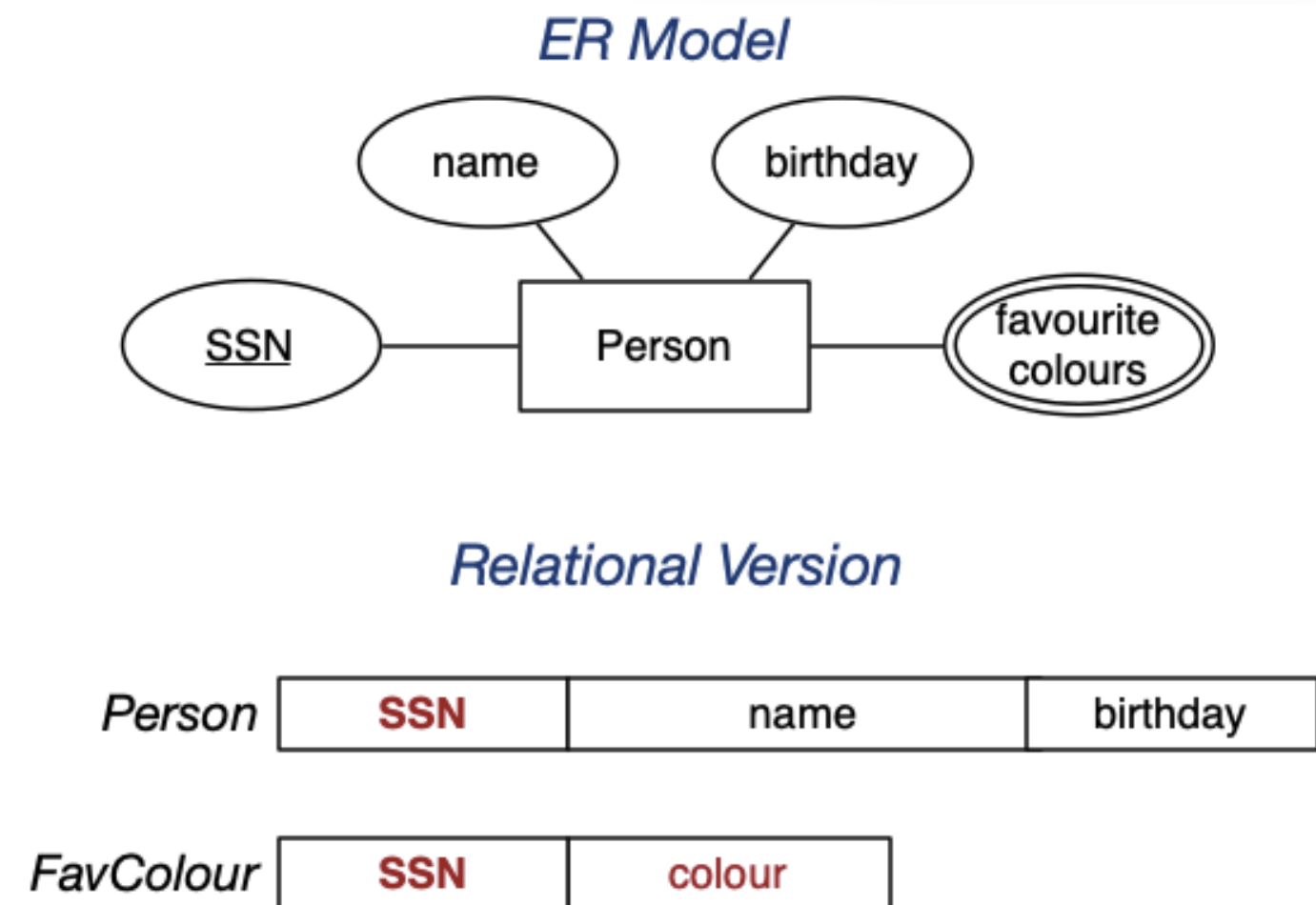
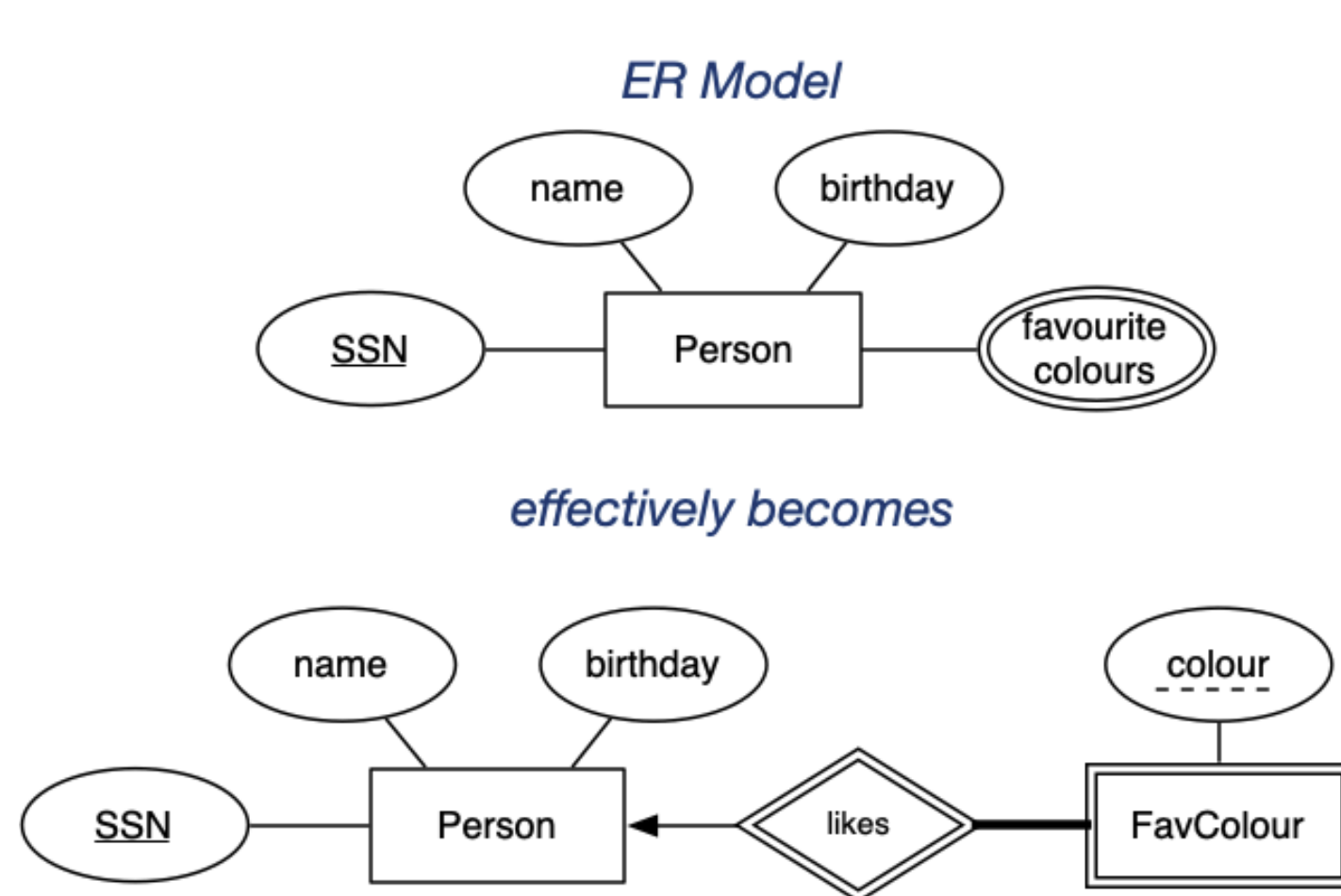


Week 2 Key Concepts

- **Multi-valued attributes**

- can be represented as a **weak entity**, with the discriminator being a single element of the multi-valued attribute

ER → Relational Model: Mapping Relationships



Week 2 Key Concepts

ER → Relational Model: Mapping Relationships

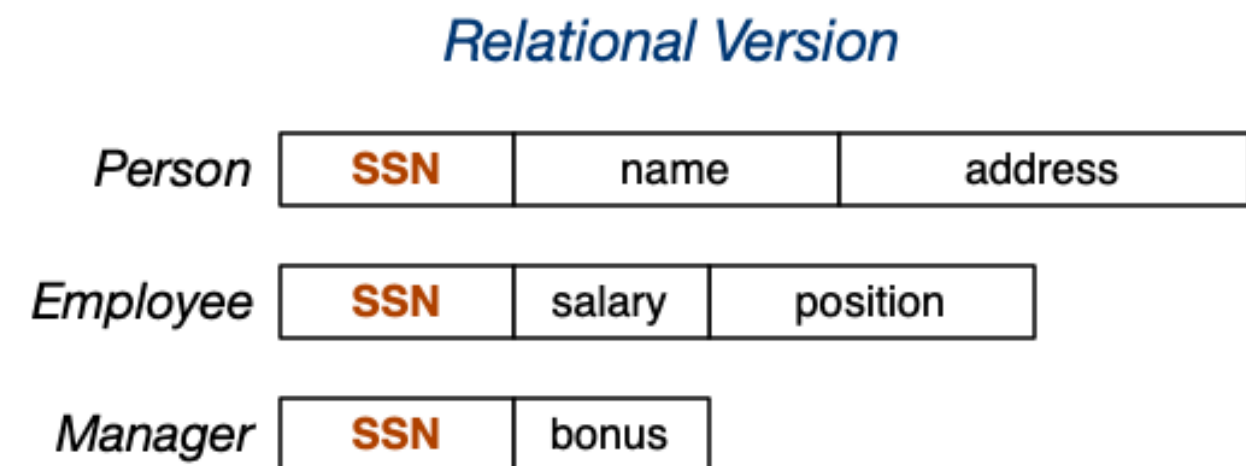
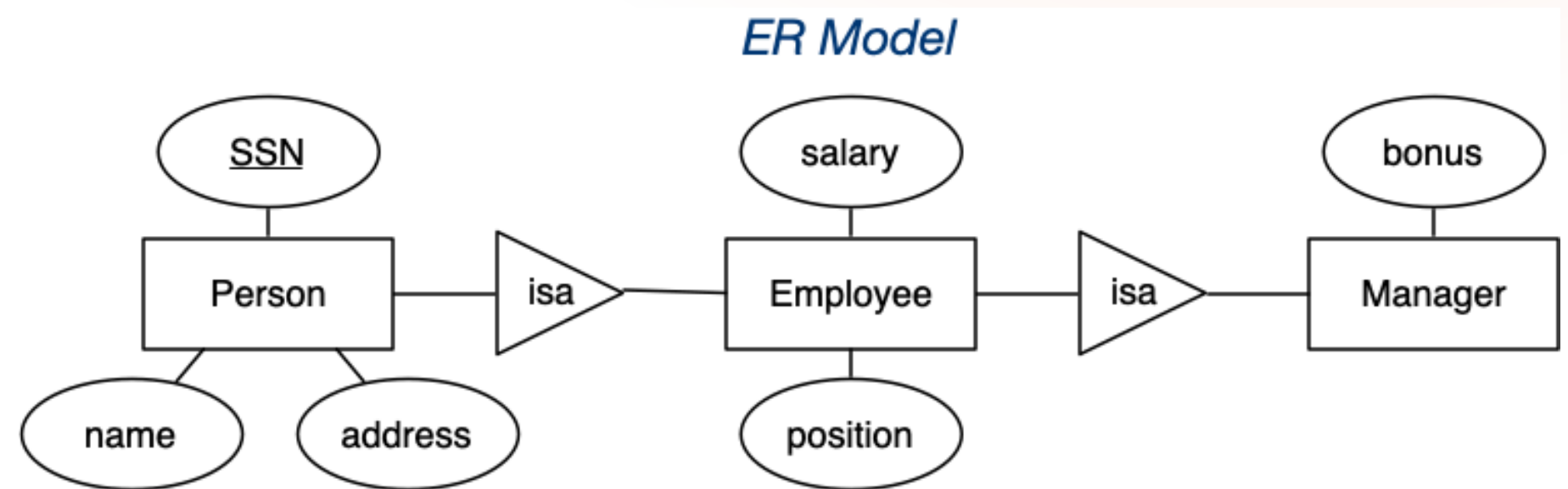
- **Subclass mapping**
 - **ER style** (make tables for each subclass, with their subclass-specific attributes, and foreign key to superclass)
 - **OO style/Object Oriented Style** (ER style but subclass tables also include inherited attributes of superclass)
 - **Single table with nulls** (make one table to represent all the subclasses)

Week 2 Key Concepts

ER → Relational Model: Mapping Relationships

- **Subclass mapping - ER style**

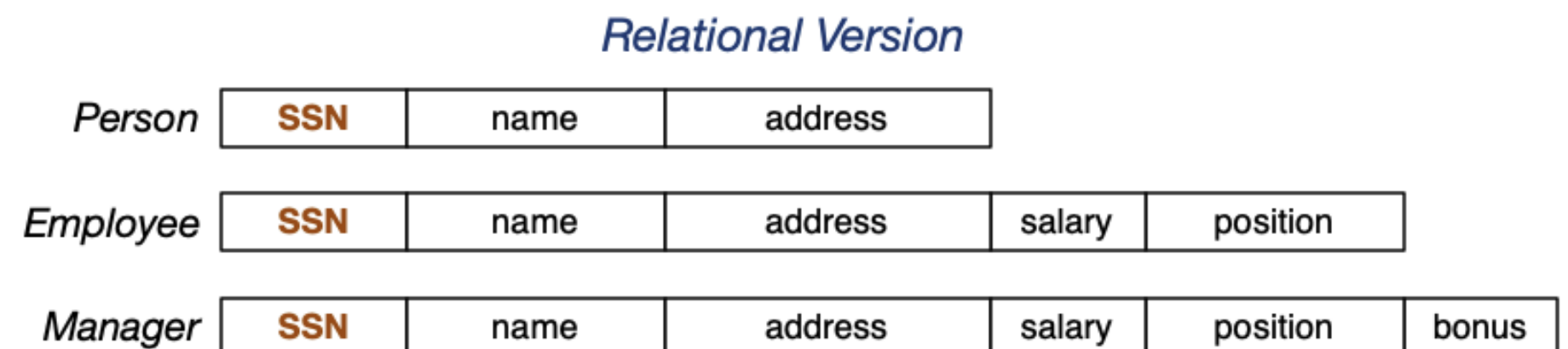
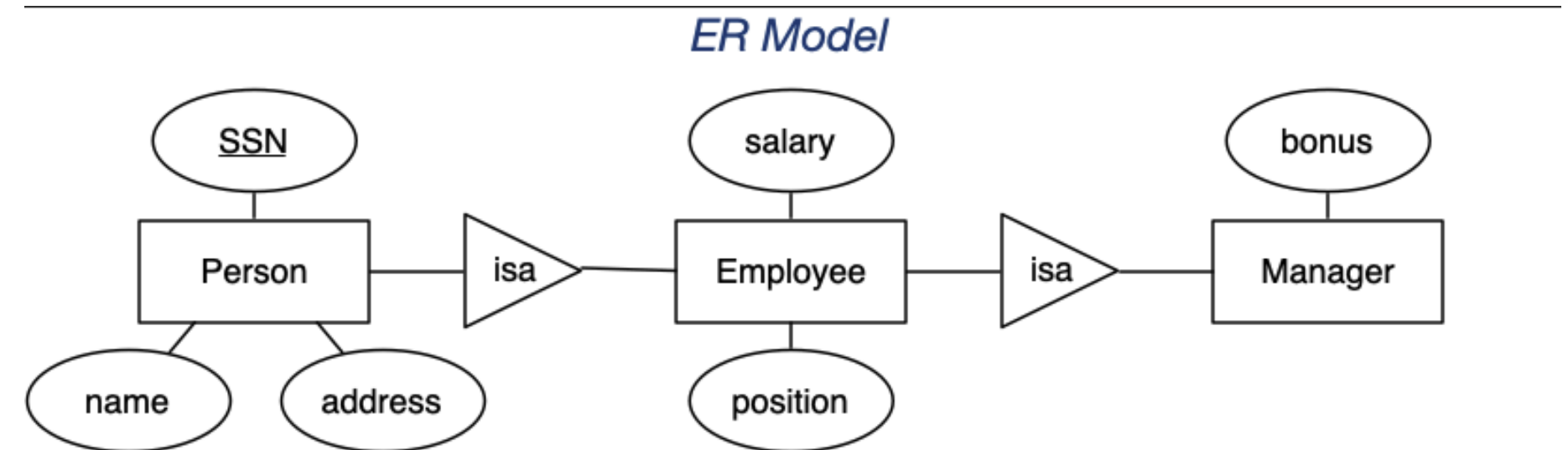
- each entity gets its own relation holding only its additional specific attributes plus a primary key joining all relations together



Week 2 Key Concepts

ER → Relational Model: Mapping Relationships

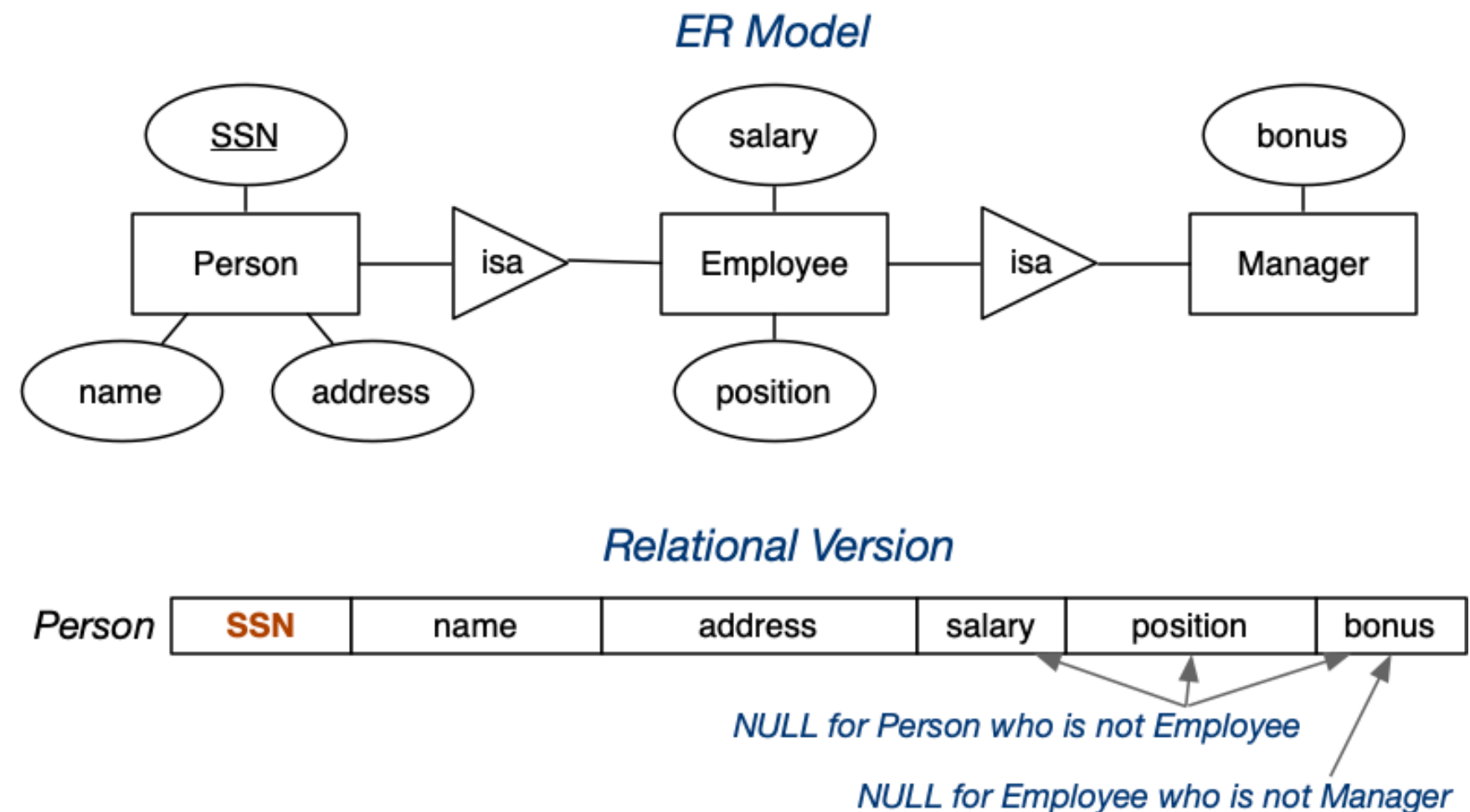
- **Subclass mapping - OO Style**
 - basically the same as ER-style subclass mapping but now you also **add the inherited attributes** as well
 - more self-contained, you don't need to refer to the superclass for attributes
 - has lots of redundant repeated data



Week 2 Key Concepts

ER → Relational Model: Mapping Relationships

- **Subclass mapping - Single table with NULLs**
 - Put all attributes into one table
 - Try to **use attributes** in order to represent which rows/instances are part of which subclass or subclasses
 - If an instance is not in a subclass, leave those subclass-specific attributes as NULL.
 - If very little instances in the subclasses, lots of wasted data.



Tutorial Questions

Now let's try some questions :D

- Q2
- Q3
- Q4

Week 2 Key Concepts

SQL data definition language

- Tables are created by the following syntax:

```
CREATE TABLE Table_Name (  
    attribute_1 domain_1 att1_constraints,  
    attribute_2 domain_2 att2_constraints,  
    ...  
    -- table-level constraints (multiple attributes)  
    CONSTRAINT constraint_name_1 table_constraint_1,  
    ...  
    CONSTRAINT constraint_name_n table_constraint_n -- No comma at end  
); -- Semicolon at end is important to note end of statement
```

- **Domains** determine the data types, whilst **constraints** ensure data integrity.
- The constraints only apply per row, so you can't really make a constraint dependent on another row.
- Entity names tend to be pluralised in SQL (Person -> People).
- You may also need to alter the table after creating it (e.g. add a foreign key for a table that didn't exist initially)

```
ALTER TABLE table_name ADD column_name data_type
```

Week 2 Key Concepts

SQL data definition language

Common SQL **domains/data types**:

- **INTEGER**
- **SERIAL** (Integer + sequence that automatically increments when not given a value)
- **TEXT** (infinite length)
- **CHAR(10)** (fixed length of 10 chars)
- **VARCHAR(10)** (variable length, max 10 chars)
- **NUMERIC/DECIMAL**
- **NUMERIC(20,2)** for 20 digit numbers, 2 being decimal places
- **DATE, TIME, DATETIME, TIMESTAMP**
- **FLOAT** (less memory than **NUMERIC**, less accurate)

Common SQL **constraints**

- **UNIQUE**: Ensures value is unique within table
- **NOT NULL**: Ensures value cannot be **NULL**.
- **CHECK (condition)**: Checks if *condition* is passed when inserting tuples into the database.
- **DEFAULT value**: Provide a default *value* if one isn't supplied
- **PRIMARY KEY (atts)**: Creates the primary key of the table.
- **FOREIGN KEY (atts) REFERENCES Table(atts)**: Creates a foreign key. **Note**: ensure you create tables in an order where all foreign keys can be made (i.e. Table has already been made beforehand).

Can also make your own domain

```
CREATE DOMAIN domain_name AS data_type constraint(s);  
-- Custom Domain for TFN, string of 11 characters, must fit format of XXX-XXX-XXX  
CREATE DOMAIN TAX_FILE_NUM AS CHAR(11) CHECK (VALUE ~ '^[0-9]{3}-[0-9]{3}-[0-9]{3}$');
```

Week 2 Key Concepts

SQL data definition language

Primary keys and foreign keys

- There are two different ways to provide a table primary/foreign keys.
- For single-attribute keys, both methods work, but for composite keys, only one specific method works.

```
create table R (  
    a integer primary key,  
    b char(10) references S(y),  
);  
-- Equivalent  
create table R (  
    a integer,  
    b char(10),  
    primary key (a),  
    foreign key (b) references S(y)  
);  
  
-- Only way to have composite keys  
create table R (  
    a integer,  
    b char(10),  
    primary key (a, b),  
    foreign key (a, b) references S(x, y)  
);
```

Tutorial Questions

Now let's try some more questions :D

- Q5
- Q13
- Q14
- Q18
- Q17

Reminders for the Week

Have you done these things???

☒ Quiz 2 (Due this Friday 11:59pm)

☒ Start Assignment 1